Trabalho Prático II
Arquivo de dados por hashing e índices B+Tree

Introdução a Bancos de Dados — ICC205 — 2026/02
Instituto de Computação — Universidade Federal do Amazonas
Docente: Altigran Soares da Silva

Aluno: Luiggy Alves (luiggy.alves@icomp.ufam.edu.br)

Outubro de 2026

Resumo

Este relatório descreve a implementação, em C++, de um arquivo de dados paginado organizado por hashing estático (Parte 1). Descreve também dois índices B+Tree em memória secundária sobre o mesmo arquivo (Parte 2): um primário, sobre o ID, e um secundário, sobre o Título. O relatório compara as duas partes nas consultas oficiais, em número de blocos lógicos lidos pela camada *blockio*. Com BLOCK_SIZE = 4096, a amostra (204.285 registros) ocupa 41.637 blocos no arquivo de dados. A busca exata por ID custa 2 blocos pelo hashing e 4 blocos pela B+Tree (altura 3). A busca por título cai de 41.637 blocos (varredura) para 5 blocos com o índice secundário. O intervalo [200000, 260000] cai de 41.637 para 3.070 blocos. Com blocos de 8192 bytes, o arquivo de dados cai para 19.298 blocos e as duas árvores perdem um nível.

1. Visão geral da implementação

O programa `upload` lê o CSV e grava os registros no arquivo de dados por hashing. Depois, na mesma execução, ele constrói os dois índices. Os outros seis programas só leem esses três arquivos. Todo acesso aos arquivos de dados e de índice passa por `blockio::BlockFile`; nenhum programa usa `fstream` ou `mmap` sobre eles. O CSV é lido pelo `CsvReader` fornecido e só pelo `upload`.

arquivo (em /data/db)	conteúdo	blocos (4096 B)	blocos (8192 B)
artigos.dat	arquivo de dados: bloco 0 = cabeçalho; 1..N = buckets; depois, overflow	41.637	19.298
idx_id.bpt	índice primário (B+Tree de ID); raiz no bloco 0	913	448
idx_titulo.bpt	índice secundário (B+Tree de Título); raiz no bloco 0	5.701	2.807

módulo	responsabilidade
include/datafile.hpp, src/datafile.cpp	layout de registro e de bloco, hashing, overflow, busca, varredura
include/bptree.hpp, src/bptree.cpp	B+Tree genérica em disco (chave de tamanho fixo comparada com memcmp)
include/config.hpp	parâmetros de projeto (prefixo de 64 bytes, ocupação alvo 0,80) e caminhos
src/upload.cpp e os 6 programas de consulta	programas pedidos no enunciado
tests/	testes unitários (make test) e de ponta a ponta (make test-integration)
tools/analise.cpp, scripts/	experimentos complementares, execução das consultas oficiais, relatório

Número de registros: o CSV tem 204.287 linhas. Duas delas (IDs 368004 e 424931) estão truncadas no próprio arquivo e terminam no meio de um campo entre aspas. O `CsvReader` as descarta. Por isso a carga tem 204.285 registros, e não os 204.279 citados no enunciado. Todos os números deste relatório se referem aos 204.285 registros efetivamente carregados.

7.1 Decisões de projeto

7.1.1 Layout do registro

O layout adotado é de **tamanho variável com indicadores de comprimento**. O registro tem uma parte fixa de 39 bytes: tamanho total (2), ID (4), ano (4), citações (4), data/hora (19 bytes, fixo) e os comprimentos dos três textos (3 × 2). Em seguida vêm os bytes de título, autores e snippet, sem preenchimento. Medidas no CSV (em bytes UTF-8): título médio 72,9 (máx. 302), autores médio 40,7 (máx. 791) e snippet médio 377,1 (máx. 1036).

Justificativa. Um registro de tamanho fixo com os tamanhos máximos do enunciado ocuparia ≈ 1505 bytes. Caberiam então só 2 registros por bloco de 4096 B, e a varredura leria ≈ 102.143 blocos. O tamanho fixo também obrigaria a truncar os autores de 552 registros (mais de 150 bytes) e os campos de outros 2. Com tamanho variável, o registro médio tem 529,6 bytes. Cabem ≈ 7 registros médios por bloco, e o maior registro (1323 bytes) cabe folgado nos 4080 bytes úteis. Assim, **nenhum campo é truncado** (0 registros truncados na carga). O custo do tamanho variável é que a quantidade de registros por bloco deixa de ser constante (de 0 a 11 por bloco).

Bloco de dados: cabeçalho de 16 bytes (*next* = próximo bloco da cadeia, *bucket*, *count*, *used*), seguido dos registros colados uns aos outros. Não há remoção nem atualização, então o endereço de um registro (RID) é o par (bloco, deslocamento no bloco), com 4 + 2 = 6 bytes. É esse RID que os índices guardam.

7.1.2 Hashing

Função: h(ID) = ID mod N, com N primo. Os IDs vão de 5 a 1.549.142. Como a amostra pega um registro a cada cinco, os IDs têm padrões aritméticos: por resíduo mod 5, a contagem é 44.339, 43.464, 38.955, 44.784, 32.743. Um N primo não tem divisores em comum com esses padrões, e o resto da divisão espalha os IDs de modo uniforme. A seção 7.6 mostra que a distribuição medida é praticamente a de Poisson.

Número de buckets: N é calculado na carga. O `upload` faz uma primeira passada no CSV para somar os tamanhos dos registros e escolhe o menor primo ≥ bytes / (bytes úteis × 0,80). A meta é ocupar 80 % dos blocos primários. O resultado é N = 33.151 com 4096 B e N = 16.547 com 8192 B. O valor 0,80 foi escolhido na fase de projeto, com uma simulação em Python sobre o CSV para ocupações de 0,6 a 0,9. Ocupações menores reduzem o overflow, mas aumentam o arquivo e, portanto, o custo de toda varredura. Ocupações maiores fazem o contrário.

Colisões e overflow: cada bucket tem 1 bloco primário (bucket b → bloco 1 + b). Registros que colidem no mesmo bucket ficam juntos no bloco enquanto houver espaço. Quando o registro não cabe, o `upload` aloca um bloco de overflow no fim do arquivo e o liga ao último bloco da cadeia pelo campo *next*. Isso é encadeamento separado por bucket. Durante a carga, o último bloco de cada cadeia fica em memória, então cada inserção lê 1 bloco e grava 1 (ou 3, se abrir um overflow). A busca por ID percorre a cadeia só até achar o ID, que é chave.

Cabeçalho: N é guardado no bloco 0, junto com contadores. O `findrec` precisa de N para calcular h(ID), então toda busca lê o cabeçalho antes do bucket. Esse bloco a mais aparece em todas as medições do `findrec` e é discutido na seção 7.4.

7.1.3 B+Tree: estrutura do nó e ordem

Uma única implementação (classe `BPlusTree`) atende aos dois índices. Cada nó ocupa um bloco e começa com um cabeçalho de 32 bytes: magic, folha/interno, n, tamanho da chave, *next* e, só na raiz, os metadados (altura, total de entradas, primeira folha, BLOCK_SIZE). A folha guarda n pares (chave, RID de 6 bytes). O nó interno guarda n chaves e n + 1 ponteiros de 4 bytes (número do bloco). A **raiz fica sempre no bloco 0**. Quando ela se divide, as metades vão para blocos novos e o bloco 0 é regravado como a nova raiz. Assim a busca não lê um bloco de cabeçalho à parte: os metadados estão na raiz, que seria lida de qualquer modo.

Cálculo da ordem (B = BLOCK_SIZE, k = tamanho da chave):

```
capacidade da folha  Lf = floor((B - 32) / (k + 6))
chaves no interno    Li = floor((B - 32 - 4) / (k + 4))     ordem p = Li + 1 filhos
```

índice	chave (k)	Lf (4096)	p (4096)	Lf (8192)	p (8192)
primário (ID)	4 bytes (ID em big-endian, bit de sinal invertido)	406	508	816	1020
secundário (Título)	70 bytes = 64 do título + 6 do RID	53	55	107	111

As chaves são comparadas com `memcmp`. O ID é gravado em big-endian com o bit de sinal invertido, para que a ordem dos bytes seja a ordem numérica. Inserção: a busca desce da raiz até a folha e guarda o caminho. Se a folha estourar, ela é dividida ao meio e a primeira chave da metade direita é *copiada* para o pai. Se o nó interno estourar, a chave do meio *sobe* para o avô. As folhas são encadeadas pelo campo *next*, e é esse encadeamento que o `seekrange` percorre. Toda leitura e escrita de nó passa pelo blockio, sem cache.

7.1.4 Índice secundário: chave de título e duplicatas

Duplicatas: a chave do índice secundário é composta, (título, RID). Como o RID é único, toda chave é única, mesmo quando vários registros têm o mesmo título. O CSV tem 192.866 títulos distintos para 204.285 registros. Os registros de mesmo título ficam lado a lado nas folhas, ordenados pelo RID. A busca monta a chave (título, RID 0), que é menor que qualquer entrada com aquele título, posiciona-se na primeira entrada ≥ essa chave e avança pelas folhas enquanto o título for igual. Não é preciso tratar duplicatas à parte (listas de RIDs, buckets de ponteiros), e a árvore continua com chaves únicas.

Prefixo de 64 bytes: para a ordem poder ser calculada a partir de BLOCK_SIZE, a chave precisa ter tamanho fixo. Usar os 300 bytes do enunciado reduziria a ordem a ≈ 13 e a árvore teria 5 ou 6 níveis. A chave guarda então os primeiros 64 bytes do título. O título completo continua no arquivo de dados e é conferido no registro lido, então o resultado é exato. O custo é que títulos diferentes com o mesmo prefixo de 64 bytes viram *candidatos* e custam uma leitura de dados cada. A escolha de 64 veio desta medição sobre os títulos distintos do CSV:

prefixo (bytes)	prefixos compartilhados por títulos diferentes	maior grupo	Lf (4096)	p (4096)
32	1.989	17	92	97
48	757	13	67	71
64	457	3	53	55
100	82	2	36	37
128	26	2	29	30
200	0	1	19	20
300	0	1	13	14

Os títulos distintos são 192.866. Com 64 bytes, a ordem é 55 e a altura prevista é 4. Os poucos prefixos compartilhados custam, em média, 1,0602 leituras de dados por busca (seção 7.6).

7.1.5 Construção dos índices

Os índices são construídos dentro do `upload`, logo após o arquivo de dados. O comando é o mesmo da carga: `./bin/upload /data/artigo-amostra.csv`. O arquivo de dados é varrido em ordem física (41.636 leituras com 4096 B), porque é dessa varredura que saem os RIDs. Cada registro gera uma inserção em cada árvore. Custos medidos (blocos):

estrutura	lidos (4096)	escritos (4096)	lidos (8192)	escritos (8192)
arquivo de dados (carga por hashing)	204.285	245.923	204.285	223.584
varredura do arquivo de dados para os índices	41.636	0	19.297	0
índice primário	504.299	206.107	407.754	205.179
índice secundário	742.850	215.681	604.683	209.895

7.2 Previsão antes da medição

A previsão abaixo foi registrada em `docs/previsao.md` com o código já compilado, mas **antes** da primeira execução do `upload` e das consultas. Ela usa só o layout e modelos analíticos: distribuição

de Poisson para os buckets e ocupação ln 2 ≈ 69 % para B+Trees construídas por inserção em ordem aleatória. O conteúdo é reproduzido sem alterações.

Registrada em 12/10/2026, **antes** da primeira execução do `upload` e das consultas oficiais. Os valores abaixo saem só do layout (código já escrito, mas não executado) e de modelos analíticos simples. A comparação com o que foi medido está no relatório (Seção 7.4).

Entradas do modelo

- n = 204.279 registros (número informado no enunciado).
- Tamanho médio do registro no layout adotado: 39 bytes fixos + título (72,9) + autores (40,7) + snippet (377,0) ≈ **529,6 bytes**. As médias dos campos foram medidas no CSV.
- Bytes úteis por bloco de dados = BLOCK_SIZE − 16 (cabeçalho do bloco).
- N = ⌈n · 529,6 / (bytes úteis · 0,80)⌉, arredondado para o primo seguinte.
- Número de registros por bucket ~ Poisson(λ = n/N). O bucket transborda se receber mais registros do que cabem no bloco: c = ⌊bytes úteis / 529,6⌋.
- B+Tree construída por inserções em ordem aleatória de chave (a ordem física do arquivo hash). A ocupação média esperada dos nós é ln 2 ≈ 69 % (Yao, 1978).

Parâmetros previstos

Grandeza	4096 B	8192 B
registros por bloco (registro médio)	7,7 → 7	15,4 → 15
N (buckets)	≈ 33.149	≈ 16.547
λ = registros por bucket	6,16	12,35
P(bucket transbordar) = P(X ≥ c+1)	0,28	0,18
blocos de overflow ≈ N · P	≈ 9.240	≈ 3.010
total de blocos do arquivo de dados	≈ 42.390	≈ 19.550
índice primário: folha / ordem do interno	406 / 508	816 / 1020
índice primário: folhas (69 %)	≈ 726	≈ 361
índice primário: altura	**3**	**2**
índice secundário: folha / ordem	53 / 55	107 / 111
índice secundário: folhas (69 %)	≈ 5.560	≈ 2.750
índice secundário: altura	**4**	**3**

Com folhas 100 % cheias, o índice primário de 4096 B teria 504 folhas e caberia sob uma raiz de ordem 508 (altura 2). Com inserção aleatória (≈ 69 %), ele precisa de ≈ 726 folhas. Por isso a previsão é de altura 3.

Blocos lidos previstos por operação (BLOCK_SIZE = 4096)

Operação	Caso	Índice	Dados
findrec	ID existente	–	2 (cabeçalho + bucket). Será 3 se o registro estiver no overflow (≈ 10 % dos registros).
findrec	ID inexistente	–	1 + comprimento da cadeia: 2 (72 %) ou 3 (28 %)
findtitle	qualquer título	–	todos os blocos ≈ 42.390
findrange	qualquer intervalo	–	todos os blocos ≈ 42.390
seek1	ID existente	3	1
seek1	ID inexistente	3	0
seek2	título único	4 (ou 5 se cruzar a fronteira de uma folha)	1
seek2	título com k repetições	4 + ⌈k/36,7⌉ − 1 (podendo ser +1)	k (45 e 26)
seek2	título inexistente	4 (ou 5)	0
seekrange	[1, 50] (10 registros)	3	10
seekrange	[1000, 1100] (20)	3	20
seekrange	[500000, 500500] (99)	3 ou 4	99

seekrange	[200000, 260000] (3.052)	3 + ⌈3052/281⌉ ≈ 14	3.052
seekrange	[1600000, 1700000] (0)	3	0

A quantidade de registros de cada intervalo foi contada no CSV.

Com 8192 B, os custos previstos são: findrec 2; seek1 2 + 1 = 3; seek2 3 + 1; varreduras ≈ 19.550 blocos; seekrange [200000, 260000] ≈ 2 + ⌈3052/566⌉ ≈ 8 no índice.

Custo previsto de construção (4096 B)

- Arquivo de dados: N escritas para criar os buckets vazios, mais 1 leitura e 1 escrita por registro, mais 1 escrita extra por bloco de overflow. Total previsto: ≈ 204 mil leituras e ≈ 247 mil escritas.
- Índice primário: ≈ altura leituras por inserção (≈ 2,9 em média, porque a árvore cresce durante a carga), ou seja, ≈ 590 mil leituras. Escritas: ≈ 1 por inserção, mais 2 por divisão, ou seja, ≈ 206 mil escritas.
- Índice secundário: ≈ 3,9 leituras por inserção (≈ 800 mil). Escritas: ≈ 204 mil + 2 × 5.600 divisões ≈ 216 mil.

7.3 Medição (BLOCK_SIZE = 4096)

Os números abaixo são exatamente os produzidos por `bash scripts/run_experiments.sh`, que executa todas as consultas de `queries/oficial.txt`. As linhas RESUMO brutas estão em `docs/resultados/consultas_4096.txt`. Totais de blocos: arquivo de dados = **41.637** (1 cabeçalho + 33.151 primários + 8.485 overflow); índice primário = **913** (910 folhas + 3 internos, altura 3); índice secundário = **5.701** (5.566 folhas + 135 internos, altura 4).

Busca por ID

ID	existe	findrec: dados	seek1: índice	seek1: dados	seek1: total
5	sim	2	3	1	4
316639	sim	2	3	1	4
663752	sim	2	3	1	4
1014283	sim	2	3	1	4
1483294	sim	2	3	1	4
1	não	3	3	0	3
2000000	não	2	3	0	3

Busca por título

título	registros	findtitle: dados	seek2: índice	seek2: dados	seek2: total
Design and implementation of an immersive virtual reality system based on a smartphone platform.	1	41.637	5	1	6
A rotating-magnet based wireless electrical power transfer for biomedical implants	1	41.637	4	1	5
Review of particle physics particle data group	45	41.637	6	45	51
Two-dimensional gas of massless Dirac fermions in graphene	26	41.637	5	26	31
Um titulo que nao existe na base	0	41.637	4	0	4

Busca por intervalo

intervalo	registros	findrange: dados	seekrange: índice	seekrange: dados	seekrange: total
[1, 50]	10	41.637	3	10	13
[1000, 1100]	20	41.637	4	20	24
[500000, 500500]	99	41.637	3	99	102

[200000, 260000]	3.052	41.637	18	3.052	3.070
[1600000, 1700000]	0	41.637	3	0	3

Totais: arquivo de dados 41.637 blocos; índice primário 913 blocos; índice secundário 5.701 blocos. Os programas da Parte 2 informam também o total de blocos do arquivo de dados.

Estatísticas da carga (saída do `upload`): 204.285 registros; registros por bloco: média 4,91 (mín. 0, máx. 11); taxa de ocupação dos blocos de dados 63,4 %; buckets com overflow 8.476 (25,6 %); maior cadeia 3 blocos.

7.4 Comparação entre previsão e medição

grandeza	previsto	medido	situação
N (buckets)	≈ 33.149	33.151	acerto (n = 204.285, não 204.279)
blocos de overflow	≈ 9.240	8.485	medido 8 % menor
total de blocos do arquivo de dados	≈ 42.390	41.637	medido 1,8 % menor
findrec, ID existente	2 (3 em ≈ 10 %)	2 nos 5 IDs; média geral 2,080	acerto
findrec, ID inexistente	2 (72 %) ou 3 (28 %)	3 (ID 1) e 2 (ID 2000000); média 2,256	acerto
findtitle / findrange	≈ 42.390	41.637	igual ao total, como previsto
altura do índice primário	3	3	acerto
folhas do índice primário	≈ 726 (69 %)	910 (55,3 %)	**divergência**: ver (a)
seek1 (existente / inexistente)	3 + 1 / 3 + 0	3 + 1 / 3 + 0	acerto
altura e folhas do índice secundário	4 e ≈ 5.560	4 e 5.566	acerto
seek2, título único	4 (ou 5) + 1	5 + 1 e 4 + 1	um caso com 5: ver (b)
seek2, 45 repetições	4 + 1 (ou + 2) + 45	6 + 45	acerto
seek2, 26 repetições	4 (ou 5) + 26	5 + 26	acerto
seek2, inexistente	4 (ou 5) + 0	4 + 0	acerto
seekrange [1000, 1100]	3 + 20	4 + 20	ver (c)
seekrange [200000, 260000]	≈ 14 + 3.052	18 + 3.052	**divergência**: ver (a)
construção do índice primário (leituras / escritas)	≈ 590 mil / ≈ 206 mil	504.299 / 206.107	leituras menores: ver (d)
construção do índice secundário (leituras / escritas)	≈ 800 mil / ≈ 216 mil	742.850 / 215.681	ver (d)

A previsão acertou a forma de todos os custos. Também acertou o valor exato nas buscas por ID e nas varreduras. As divergências são explicadas a seguir.

(a) Ocupação das folhas do índice primário: 55,3 % em vez de 69 %. A previsão supôs que as chaves chegam em ordem aleatória. No `upload`, porém, os índices são construídos varrendo o arquivo de dados em ordem física, bucket por bucket. O bucket b guarda os IDs ≡ b (mod N), ou seja, b, b + N, b + 2N, ... Ao passar do bucket b para o b + 1, cada uma das ≈ 47 faixas [jN, (j+1)N) da árvore recebe um ID um pouco maior que o anterior daquela faixa. Dentro de cada faixa a inserção é crescente. Com chaves crescentes, a folha dividida fica com metade das entradas e nunca mais recebe inserções, então a ocupação tende a 50 %. Só os ≈ 8,0 % de registros nos blocos de overflow, varridos no fim, chegam fora dessa ordem e preenchem parte das folhas. O experimento D da ferramenta `analise` confirma a explicação. Ele reconstrói o índice primário com as mesmas 204.285 entradas em três ordens:

```
[D] indice primario construido com 3 ordens de insercao diferentes
    ordem                              altura   folhas internos ocup.folha     leituras     escritas
    fisica do arquivo hash (upload)         3      910        3      55.3%       504299       206107
    crescente de ID (ordem do CSV)          3     1001        4      50.3%       508614       206291
    aleatoria                               3      717        3      70.2%       476648       205721
```

A ordem aleatória reproduz a previsão (717 folhas, 70,2 %). A ordem crescente dá 50,3 %, e a

ordem física do hash fica entre as duas (55,3 %). A altura continua 3 nos três casos, então o `seek1` não muda. Muda o número de folhas que o `seekrange` percorre. No intervalo [200000, 260000] há 3.052 registros a ≈ 224 por folha, e não ≈ 281: são 16 folhas visitadas (a primeira vem da descida e as outras 15 pelo ponteiro *next*), total de 3 + 15 = 18 leituras no índice.

(b) seek2 com 5 leituras para um título único. O rastro abaixo (TP2_TRACE=1) mostra a causa. A chave de busca (prefixo, RID 0) é menor que a entrada procurada. Essa entrada é a *primeira* de sua folha e, por isso, também é o separador no pai. A descida vai para o filho da esquerda, cujas entradas são todas menores. O cursor chega ao fim dessa folha e precisa ler a seguinte. Isso acontece sempre que o título procurado é o primeiro de uma folha. Na amostra de 5.000 títulos do experimento B, ocorreu em 6,34 % deles (média de 4,063 leituras no índice).

```
[trace] data/db_4096/idx_titulo.bpt: le bloco 0 (no interno, 3 chaves, raiz)
[trace] data/db_4096/idx_titulo.bpt: le bloco 3996 (no interno, 29 chaves)
[trace] data/db_4096/idx_titulo.bpt: le bloco 3431 (no interno, 44 chaves)
[trace] data/db_4096/idx_titulo.bpt: le bloco 5330 (folha, 27 chaves)
[trace] data/db_4096/idx_titulo.bpt: le bloco 3 (folha, 28 chaves)
[trace] data/db_4096/artigos.dat: le bloco 6 (registro no deslocamento 16)
registros encontrados: 1
blocos lidos no indice secundario: 5 (altura 4)
blocos lidos no arquivo de dados : 1
```

(c) Intervalos que cruzam a fronteira de uma folha. [1000, 1100] tem só 20 registros, mas eles começam no fim de uma folha e terminam na seguinte: 3 leituras na descida + 1 pelo *next* = 4. O mesmo vale para os títulos repetidos: as 45 entradas de "Review of particle physics particle data group" fazem o percurso tocar 3 folhas: 6 = 3 nós internos + 3 folhas.

```
[trace] data/db_4096/idx_id.bpt: le bloco 0 (no interno, 1 chaves, raiz)
[trace] data/db_4096/idx_id.bpt: le bloco 510 (no interno, 431 chaves)
[trace] data/db_4096/idx_id.bpt: le bloco 1 (folha, 204 chaves)
[trace] data/db_4096/artigos.dat: le bloco 1004 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1009 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1014 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1019 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1024 (registro no deslocamento 16)
[trace] data/db_4096/idx_id.bpt: le bloco 34 (folha, 204 chaves)
[trace] data/db_4096/artigos.dat: le bloco 1029 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1034 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1039 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1044 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1049 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1054 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1059 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1064 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1069 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1074 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1079 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1084 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1089 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1094 (registro no deslocamento 16)
[trace] data/db_4096/artigos.dat: le bloco 1099 (registro no deslocamento 16)
registros encontrados: 20
blocos lidos no indice primario : 4 (altura 3; folhas com entradas visitadas: 2)
blocos lidos no arquivo de dados: 20
```

(d) Custo de construção. A previsão usou ≈ 2,9 leituras por inserção no índice primário. O medido foi 2,47, porque a árvore teve altura 1 e 2 durante parte da carga e chegou a 3 só depois de 508 folhas. As escritas ficaram próximas do previsto: 1 por inserção, mais 2 por divisão. O índice secundário leu 3,64 blocos por inserção, contra 3,9 previstos, pelo mesmo motivo.

(e) Overflow menor que o previsto. O modelo supôs capacidade fixa de 7 registros por bloco. Como os registros têm tamanho variável, um bloco recebe até 11 registros pequenos, e buckets com 8 ou 9 registros muitas vezes não transbordam. Por isso há 8.485 blocos de overflow, e não ≈ 9.240.

(f) O bloco de cabeçalho. O `findrec` sempre lê 1 bloco a mais (o cabeçalho, com N). Sem ele, a busca por hashing custaria 1 bloco por registro sem overflow. Esse bloco poderia ser evitado mantendo N em memória (catálogo do SGBD), mas cada programa aqui é um processo novo e lê N do disco. Mesmo pagando o cabeçalho, o hashing (2 blocos) é mais barato que a B+Tree (4 blocos) na busca exata por ID, como o enunciado antecipa.

7.5 Efeito do tamanho do bloco (4096 × 8192 bytes)

grandeza	4096	8192	variação
bytes úteis por bloco de dados	4.080	8.176	× 2,0
registros por bloco (média; capacidade p/ registro médio)	4,91; 7	10,59; 15	× 2,16
N (buckets)	33.151	16.547	× 0,50
buckets com overflow	8.476 (25,6 %)	2.750 (16,6 %)	
taxa de ocupação dos blocos de dados	63,4 %	68,4 %	
blocos do arquivo de dados (= custo de findtitle/findrange)	41.637	19.298	× 0,463
índice primário: folha / ordem	406 / 508	816 / 1020	× 2
índice primário: folhas / altura	910 / 3	447 / 2	−1 nível
índice secundário: folha / ordem	53 / 55	107 / 111	× 2
índice secundário: folhas / altura	5.566 / 4	2.774 / 3	−1 nível
blocos dos índices (primário / secundário)	913 / 5.701	448 / 2.807	

Medição das consultas oficiais com 8192 B (`docs/resultados/consultas_8192.txt`):

Busca por ID

ID	existe	findrec: dados	seek1: índice	seek1: dados	seek1: total
5	sim	2	2	1	3
316639	sim	2	2	1	3
663752	sim	2	2	1	3
1014283	sim	2	2	1	3
1483294	sim	2	2	1	3
1	não	2	2	0	2
2000000	não	2	2	0	2

Busca por título

título	registros	findtitle: dados	seek2: índice	seek2: dados	seek2: total
Design and implementation of an immersive virtual reality system based on a smartphone platform.	1	19.298	3	1	4
A rotating-magnet based wireless electrical power transfer for biomedical implants	1	19.298	3	1	4
Review of particle physics particle data group	45	19.298	3	45	48
Two-dimensional gas of massless Dirac fermions in graphene	26	19.298	3	26	29
Um titulo que nao existe na base	0	19.298	3	0	3

Busca por intervalo

intervalo	registros	findrange: dados	seekrange: índice	seekrange: dados	seekrange: total
[1, 50]	10	19.298	2	10	12
[1000, 1100]	20	19.298	2	20	22
[500000, 500500]	99	19.298	2	99	101
[200000, 260000]	3.052	19.298	9	3.052	3.061
[1600000, 1700000]	0	19.298	2	0	2

Totais: arquivo de dados 19.298 blocos; índice primário 448 blocos; índice secundário 2.807 blocos. Os programas da Parte 2 informam também o total de blocos do arquivo de dados.

Registros por bloco. Dobrar o bloco dobra os bytes úteis (o cabeçalho de 16 bytes pesa menos), e a capacidade sobe de 7 para 15 registros médios. O número de blocos do arquivo de dados caiu para 46,3 % do valor anterior, um pouco *menos* que a metade. Com $\lambda = 6{,}16$ registros por bucket, a variância relativa de Poisson ($1/\sqrt{\lambda}$) é maior que com $\lambda = 12{,}35$. Buckets grandes absorvem melhor

a flutuação, então há menos overflow (16,6 % dos buckets contra 25,6 %) e a ocupação sobe. As varreduras (`findtitle` e `findrange`) acompanham esse número e caem de 41.637 para 19.298 blocos. O `findrec` continua com 2 blocos (cabeçalho + bucket), com média geral 2,029 contra 2,080: menos registros caem no overflow. Cada leitura, porém, transfere o dobro de bytes.

Ordem e altura. A ordem é proporcional a B, então os dois índices dobram de ordem. O índice primário passa a caber em 447 folhas sob uma única raiz de ordem 1020 (altura 2). O secundário passa de 4 para 3 níveis. Logo, o `seek1` cai de 4 para 3 blocos e o `seek2` de título único de 4–5 para 3–4. No `seekrange`, as leituras de índice caem pela metade (18 → 9 no intervalo grande). As leituras de dados não mudam: continua sendo 1 bloco por registro, porque o hashing espalha IDs vizinhos por blocos diferentes. Em intervalos grandes, o custo total quase não se altera.

Consequência para a escolha do plano. Com 8192 B, a varredura fica 2,2 vezes mais barata, mas o `seekrange` não. O ponto em que o índice deixa de compensar cai de ≈ 41.637 registros (20 % da tabela) para ≈ 19.298 (9,4 %). Veja o experimento C na seção 7.6.

7.6 Casos patológicos

Busca que não encontra nada

operação	chave inexistente	blocos lidos (4096)	por quê
findrec	ID 1	3	cabeçalho + bucket 1 inteiro (o bucket 1 tem 1 bloco de overflow)
findrec	ID 2000000	2	cabeçalho + bucket sem overflow
seek1	ID 1 / 2000000	3 + 0	desce até a folha; a ausência é detectada sem tocar no arquivo de dados
findtitle	Um titulo que nao existe na base	41.637	varredura completa: só no fim se sabe que não há o título
seek2	Um titulo que nao existe na base	4 + 0	descida até a folha; nenhum candidato
findrange	[1600000, 1700000]	41.637	varredura completa
seekrange	[1600000, 1700000]	3 + 0	desce até a última folha; next = 0

Na Parte 1, uma busca sem resultado custa o mesmo que uma com resultado no caso das varreduras. No hashing, ela custa um pouco mais: precisa ler a cadeia inteira (média 2,256 contra 2,080 com sucesso, experimento A). Na Parte 2, a busca sem resultado é a mais barata possível, só a altura da árvore. O `findrec` do ID 1 (3 blocos) custa mais que qualquer busca com sucesso da lista oficial: o bucket 1 está entre os 25,6 % que transbordaram.

```
[trace] data/db_4096/artigos.dat: le bloco 0 (cabecalho)
[trace] data/db_4096/artigos.dat: le bloco 2 (primario do bucket)
[trace] data/db_4096/artigos.dat: le bloco 35575 (overflow do bucket)
blocos lidos no arquivo de dados: 3 (1 cabecalho + 2 do bucket: 1 primario + 1 overflow)
```

Índice secundário devolvendo vários registros

"Review of particle physics particle data group" tem 45 registros, e "Two-dimensional gas of massless Dirac fermions in graphene" tem 26. Como a chave é (título, RID), as entradas ficam contíguas nas folhas: o custo no índice é a altura mais as folhas extras ocupadas pelo grupo (6 e 5). O custo dominante é o do arquivo de dados: **1 bloco por registro** (45 e 26). O hashing por ID espalha registros de mesmo título por buckets diferentes, e o índice secundário é não agrupado (*unclustered*). Ainda assim, 51 blocos contra 41.637 da varredura. O mesmo vale para títulos diferentes com o mesmo prefixo de 64 bytes. Eles aparecem como candidatos, e cada candidato custa 1 leitura de dados antes de ser descartado. No experimento B, a média foi 1,0602 candidatos por título buscado.

Tamanho do intervalo e custo da B+Tree

O custo do `seekrange` para um intervalo com k registros é altura + ⌈k / entradas por folha⌉ no índice, mais k no arquivo de dados. O termo k domina rapidamente: a partir de k = 1.000, o índice

responde por menos de 1 % do total. O experimento C mede intervalos com k crescente no meio do arquivo:

```
[C] seekrange em funcao do numero k de registros do intervalo (findrange le sempre 41637 blocos)
             k          indice          dados           total   total/scan
             1               3              1               4       0.000
            10               3             10              13       0.000
           100               3            100             103       0.002
          1000               8           1000            1008       0.024
          5000              27           5000            5027       0.121
         10000              50          10000           10050       0.241
         20000              98          20000           20098       0.483
         30000             146          30000           30146       0.724
         40000             192          40000           40192       0.965
         60000             287          60000           60287       1.448  <- pior que a varredura
        100000             470         100000          100470       2.413  <- pior que a varredura

[C] seekrange em funcao do numero k de registros do intervalo (findrange le sempre 19298 blocos)
             k          indice          dados           total   total/scan
             1               2              1               3       0.000
            10               2             10              12       0.001
           100               2            100             102       0.005
          1000               4           1000            1004       0.052
          5000              14           5000            5014       0.260
         10000              24          10000           10024       0.519
         20000              48          20000           20048       1.039  <- pior que a varredura
         30000              69          30000           30069       1.558  <- pior que a varredura
         40000              91          40000           40091       2.077  <- pior que a varredura
         60000             136          60000           60136       3.116  <- pior que a varredura
        100000             224         100000          100224       5.193  <- pior que a varredura
```

Com 4096 B, o índice compensa até k ≈ 41.637 registros, cerca de 20 % da tabela. Acima disso, a varredura do `findrange` é mais barata, porque lê cada bloco uma vez só, enquanto o `seekrange` volta ao mesmo bloco de dados várias vezes, uma por registro. Com 8192 B, o limite cai para ≈ 19.298 registros. É a mesma conclusão que um otimizador de consultas tira de um índice não agrupado: só vale a pena para predicados seletivos.

Concentração de registros nos buckets

registros no bucket	buckets (medido)	Poisson λ=6,16	registros no bucket	buckets (medido)	Poisson λ=6,16
0	52	70	8	4.160	3.603
1	294	431	9	2.605	2.467
2	1.092	1.327	10	1.314	1.520
3	2.275	2.725	11	554	852
4	3.927	4.198	12	192	437
5	5.289	5.173	13	73	207
6	5.862	5.313	14	21	91
7	5.433	4.677	15	8	37

Não houve concentração anômala. A distribuição de registros por bucket segue a de Poisson com λ = 6,16, que é o esperado de uma função de hashing uniforme. O maior bucket tem 15 registros, há 52 buckets vazios (Poisson prevê 70), e nenhuma cadeia passa de 3 blocos (9 buckets com 3 blocos). O que leva 25,6 % dos buckets ao overflow não é concentração, e sim a combinação de ocupação alvo de 80 % com buckets pequenos, de ≈ 7 registros: a flutuação natural de Poisson já ultrapassa a capacidade do bloco. Com 8192 B (λ = 12,35), a mesma ocupação gera só 16,6 % de buckets com overflow e nenhuma cadeia com 3 blocos.

7.7 Declaração de uso de ferramentas de IA

Ferramenta: Claude Code (agente de programação da Anthropic, modelo Claude Opus 5.5), usado na extensão do VS Code.

O que foi solicitado: (1) na instrução inicial, o aluno pediu que a ferramenta realizasse todo o trabalho especificado no ENUNCIADO.md, com código comentado e testado. (2) Depois, pediu uma segunda verificação do cumprimento do enunciado e uma estimativa de nota pelos critérios da Seção 12. Nessa verificação, a ferramenta encontrou e corrigiu um defeito: IDs fora da faixa de 32

bits (ex.: `findrec 9999999999`) eram convertidos silenciosamente para outro número e agora são recusados (`include/args.hpp`). A correção não alterou nenhum número medido. (3) Por fim, pediu que a ferramenta redigisse esta seção a partir das informações fornecidas pelo aluno.

Partes produzidas pela ferramenta: o código da entrega foi integralmente gerado pela ferramenta. Isso inclui (1) todo o código C++ de `src/`, `include/` e `tools/`, exceto a camada `blockio` e o `CsvReader`, que vêm do template. Neste último a ferramenta só acrescentou `#include <cstdint>`, que faltava para compilar com g++ 13. (2) Os testes de `tests/`. (3) Os scripts de `scripts/`. (4) Os ajustes no Makefile e no README. (5) A previsão de `docs/previsao.md`, escrita antes da primeira execução. (6) Este relatório, gerado por `scripts/gerar_relatorio.py` a partir das saídas dos programas. As decisões de projeto da seção 7.1 foram propostas pela ferramenta, com base em medições feitas no próprio CSV.

O que foi feito posteriormente pelo aluno: o aluno revisou e fez o rastreio de todo o código gerado e preparou uma apresentação para a arguição.

Anexo — Como reproduzir os números deste relatório

```
gzip -dk data/artigo-amostra.csv.gz
make build && make test && make test-integration   # compilação e testes
bash scripts/run_experiments.sh                     # carga + consultas oficiais (4096 e 8192)
python3 scripts/gerar_relatorio.py                  # regenera este PDF

# no contêiner (BLOCK_SIZE padrão 4096):
docker build -t tp2 .
docker run --rm -v $(pwd)/data:/data tp2 ./bin/upload /data/artigo-amostra.csv
docker run --rm -v $(pwd)/data:/data tp2 ./bin/seek1 5
# BLOCK_SIZE=8192:  docker build --build-arg BLOCK_SIZE=8192 -t tp2-8k .
```